

HOSPITAL SMART QUEUE MANAGEMENT SYSTEM



UNIVERSITY OF INFORMATION TECHNOLOGY

SEPTEMBER 2026

Course Name: CST-6108 Enterprise Applications Development using Java

Project Title: Hospital Smart Queue Management System

Group Members

1. Mg La Yaung Htut

- Student ID: TNT-2384
- Email: layaunghtut@uit.edu.mm

2. Ma Hsu Shoon Wutyi

- Student ID: TNT-2346
- Email: hswuttye@uit.edu.mm

3. Ma Noo Noo Aung

- Student ID: TNT-2357
- Email: nonoaung@uit.edu.mm

4. Ma Ei Pone Chit

- Student ID: TNT-2357
- Email: eiponechit@uit.edu.mm

5. Ma Phoo Thet Ngone

- Student ID: TNT-2348
- Email: phuthetngone@uit.edu.mm

Supervisor: Dr. Daw Ei Moh Moh Aung

Department: Faculty of Computer Science

Submission Date: September 2026

Contents

Introduction	1
Project Planning and Requirement Analysis	2
System Design and Architecture	5
Development Environment Setup	8
Screen Layout and Design	10
Conclusion and Reflection	12
Appendices	14

1. Introduction

1.1 Overview

The Hospital Smart Queue System is a web-based application designed to improve the management of patients, doctors, appointments, and hospital queues. In many hospitals, patients have to wait for a long time and staff members need to manually manage queues and appointments. This can cause confusion, delays, and inefficient use of hospital resources.

The proposed system provides a centralized platform where hospital staff can manage patient information, doctor information, departments, appointments, emergency cases, and patient queues. Patients can be registered and assigned to appropriate doctors, while staff can monitor the queue and update its status.

The system is developed using Java Spring Boot, MySQL, HTML/CSS, and related web technologies. The application follows a database-driven architecture so that information can be stored and retrieved efficiently.

1.2 Project Scope

The scope of the Hospital Smart Queue System covers user account management, patient registration and management, doctor registration and management, and hospital department management. It also covers appointment management, queue number generation and management, emergency case management, and consultation and diagnosis records, along with staff management and queue status monitoring. Access throughout the system is role-based, with separate permissions for administrators, doctors, staff, and patients.

The system is mainly intended for use within a hospital environment. It focuses on improving the organization of patient queues and reducing unnecessary waiting and manual work.

1.3 Objectives

The main objective of the project is to develop a computerized hospital queue management system that reduces patient waiting time and queue confusion while providing an organized method for managing appointments. The system is intended to let hospital staff manage patients and doctors efficiently, provide emergency priority handling, and store hospital information in a centralized database, so that communication between patients, doctors, and hospital staff improves.

Beyond these core goals, the project also aims to provide a simple and user-friendly interface, reduce errors caused by manual queue management, and lay a foundation that future hospital management features can build on.

2. Project Planning and Requirement Analysis

2.1 Functional Requirements

The functional requirements describe what the system should be able to do.

User Management

The system should allow administrators to create user accounts, manage usernames and passwords, and assign user roles, covering administrator, doctor, staff, and patient accounts alike.

Patient Management

Staff should be able to register new patients, view and update their information, search for a patient by name or other details, and associate each patient with the appointments and queues they are part of.

Doctor Management

Administrators or authorized staff should be able to register doctors, assign them to departments, and record their specializations, contact information, and room numbers, as well as activate or deactivate a doctor's account when needed.

Department Management

The system should maintain the hospital's departments, including Cardiology, Dental, Emergency, Neurology, Pediatrics, Dermatology, Orthopedics, and General Medicine.

Appointment Management

Staff should be able to create appointments, assign patients to doctors, set appointment dates and times, and view or update appointment status as a visit progresses.

Queue Management

The system should create queue entries and generate queue numbers, associating each entry with the relevant patient and doctor. It should also let staff set queue priority and update queue status, giving emergency patients higher priority than routine cases.

Emergency Management

Staff should be able to record emergency cases, capturing the patient involved, the staff member handling the case, the reason for the emergency, and its current status.

Consultation Management

Doctors should be able to record consultation details against a queue entry, including the doctor involved, the diagnosis, and any consultation notes.

2.2 Non-Functional Requirements

The system should also satisfy several non-functional requirements.

Performance The system should respond to normal user requests quickly and efficiently.	Security User information and passwords should be protected, with access to system functions depending on the user's role.	Reliability The system should store information consistently and reduce errors caused by manual record keeping.
Usability The interface should be simple enough for hospital staff to use without extensive technical knowledge.	Maintainability The application should be organized into separate components, such as models, repositories, services, and controllers, so that future developers can maintain and extend it.	Scalability The system should be designed so that additional features, users, doctors, departments, and hospital branches can be added in the future.

2.3 Use Cases and User Stories

Administrator An administrator can log into the system, manage users, doctors, and departments, view hospital information, and manage overall system settings. ----- <i>User Story: As an administrator, I want to manage users and doctors so that the hospital system contains accurate information.</i>	Doctor A doctor can log into the system, view assigned appointments and the patients currently in the queue, look up patient information, and record diagnoses and consultation notes as consultations are completed. ----- <i>User Story: As a doctor, I want to see my patient queue so that I can treat patients in the correct order.</i>
--	---

Staff

Staff members can register patients, create appointments, manage queues, handle emergency cases, update appointment status, and monitor patients who are waiting for consultation.

User Story: As a staff member, I want to manage the patient queue so that patients can be served efficiently.

Patient

A patient can log into the system to view their appointment and queue information and check their current queue status.

User Story: As a patient, I want to know my queue number and status so that I understand when I may be called for consultation.

3. System Design and Architecture

3.1 System Architecture Design

The Hospital Smart Queue System uses a layered application architecture. The main layers are:

Presentation Layer

This layer provides the user interface through web pages. Users interact with the system through their browser.

Controller Layer

The controller receives requests from users and determines which operation should be performed.

Service Layer

The service layer contains the main business logic of the application. It processes operations such as creating appointments, managing queues, and handling consultations.

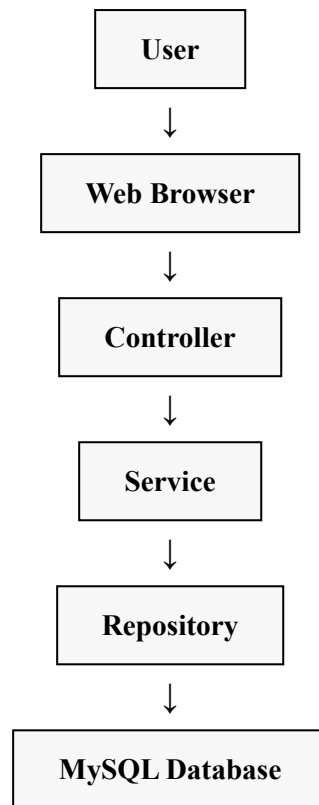
Repository Layer

The repository layer communicates with the database and performs operations such as inserting, updating, deleting, and retrieving records.

Database Layer

MySQL is used to store system data.

The basic flow is:

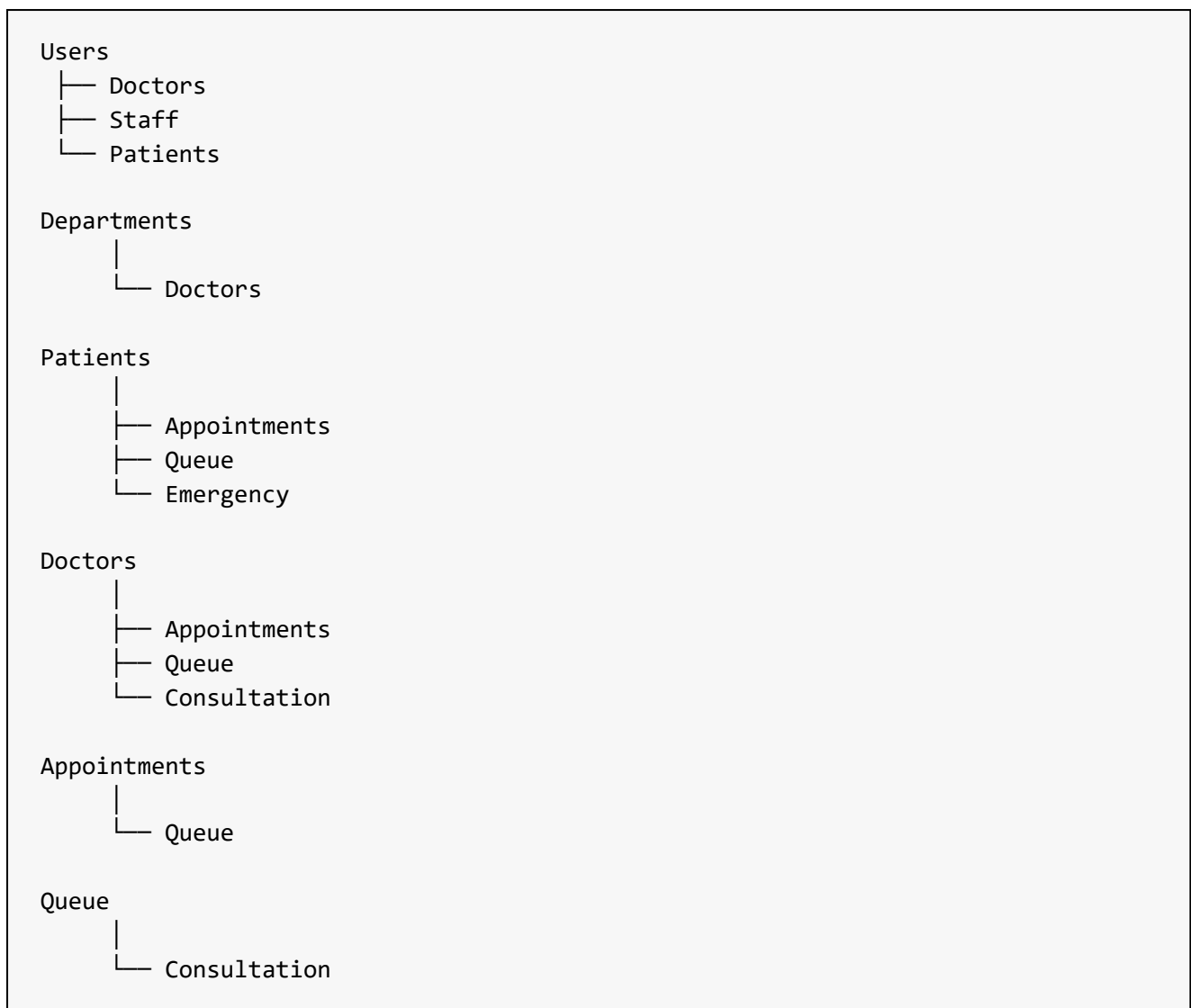


This architecture separates different responsibilities and makes the application easier to maintain.

3.2 Database Design: ER Diagram

The database contains several related entities. The main tables include users, departments, doctors, staff, patients, appointments, queue, emergency, and consultation.

The major relationships are:



For example, a patient can have multiple appointments. An appointment is associated with a doctor, and an appointment can result in a queue entry. A queue entry can then be associated with a consultation.

The database uses primary keys to uniquely identify records and foreign keys to establish relationships between tables.

4. Development Environment Setup

4.1 Tools and Technologies

The following technologies are used to develop the system:

Technology	Purpose
Java	Main programming language
Spring Boot	Backend application framework
Spring Data JPA	Database access
Hibernate	ORM framework
MySQL	Database management
Maven	Project and dependency management
HTML	Web page structure
CSS	Web page styling
JavaScript	Client-side functionality
IntelliJ IDEA	Development environment
XAMPP / phpMyAdmin	MySQL database administration
Firefox / Chrome	Application testing

4.2 Development Environment

The project is developed using IntelliJ IDEA. The backend is created using Spring Boot and managed using Maven. MySQL is used as the database server, while phpMyAdmin can be used to view and manage database records during development.

The project follows a structure similar to:

```
HospitalSmartQueueSystem
├── src
│   ├── main
│   │   ├── java
│   │   │   └── com.hospitalqueue
│   │   │       ├── controller
│   │   │       ├── model
│   │   │       ├── repository
│   │   │       └── service
│   │   └── resources
│   │       ├── templates
│   │       ├── static
│   │       └── application.properties
│   └── test
├── pom.xml
└── mvnw
```

4.3 Coding Standards

The project follows a consistent set of coding practices: meaningful class and variable names, standard Java naming conventions, and a clear separation of responsibilities into different packages. Entity classes represent database tables, repositories handle database operations, services contain business logic, and controllers handle incoming web requests. Code duplication is avoided where possible, comments are added where they improve understanding, and indentation and formatting are kept consistent throughout.

5. Screen Layout and Design

The system provides different screens depending on the user's role.

Login Screen

The login page allows users to enter their username and password. After successful authentication, the user is directed to the appropriate dashboard based on their role.

Administrator Dashboard

The administrator dashboard provides access to user management, doctor management, staff management, patient management, department management, and appointment management, giving administrators a single place to oversee the whole system.

Staff Dashboard

The staff dashboard focuses on daily hospital operations, including patient registration, appointment management, queue management, emergency cases, and patient information.

Doctor Dashboard

The doctor dashboard shows today's appointments and the current queue, along with patient information, consultation, diagnosis, and consultation notes.

Patient Dashboard

The patient dashboard can display personal information, appointment information, and the patient's queue number and status.

Queue Screen

The queue screen should clearly display information such as:

Queue Number	Patient	Doctor	Priority	Status
A001	Mg Mg	Dr. Su Su	NORMAL	WAITING
A002	Ma Ma	Dr. Su Su	NORMAL	WAITING
B001	Ko Ko	Dr. James	EMERGENCY	WAITING

Emergency cases can be visually prioritized so staff and doctors can identify them easily.

6. Conclusion and Reflection

6.1 Project Summary

The Hospital Smart Queue System was developed to improve the management of patients and hospital queues. The system combines patient management, doctor management, appointments, emergency cases, queues, and consultations into one centralized application.

The use of a relational database allows information to be stored systematically and relationships between patients, doctors, appointments, queues, and consultations to be maintained.

The system also provides a foundation for reducing manual queue management and improving hospital workflow.

6.2 Challenges and Solutions

During development, several challenges were encountered.

Database Relationships

Managing relationships between multiple tables was challenging because foreign keys must reference valid records. This was solved by inserting related data in the correct order. For example, users and doctors need to exist before dependent records such as appointments and queues can be created.

Duplicate Data

Duplicate values occurred when unique fields such as usernames or queue numbers were inserted more than once. This was handled by checking existing records before inserting new data and using unique identifiers appropriately.

Application Configuration

Connecting the Spring Boot application to MySQL required correct database configuration and compatible entity mappings. The database connection and JPA configuration were tested during development.

Queue Management

The queue system required relationships between appointments, patients, and doctors. The queue table was therefore designed to reference these records using foreign keys.

6.3 Lessons Learned

Through this project, the team developed several important skills, including Java programming, Spring Boot development, database design, SQL and MySQL, and working with JPA and Hibernate. Building the application also strengthened the team's understanding of REST and web application concepts, HTML and CSS, software architecture, debugging, foreign-key relationship management, and project planning.

The project also demonstrated the importance of testing each part of a system before connecting all components together.

6.4 Future Enhancements

Looking ahead, the system could be extended with online patient registration and online appointment booking, along with automatic queue number generation and real-time queue updates. SMS and email notifications, together with real-time emergency alerts, would keep patients and staff better informed, and a mobile application would make the system easier to reach outside the hospital.

On the administrative side, doctor availability schedules, hospital branch management, and detailed reports and statistics would support day-to-day planning, while dashboard charts and analytics would give administrators a clearer view of overall performance. Stronger authentication and password encryption, along with support for patient medical history, would further improve the security and usefulness of the system.

7. Appendices

7.1 Screen Design

Screen designs of the implemented system are presented below.

Note: The following pages show the UI/UX design references (high-fidelity wireframes) produced during the design phase of the project. They illustrate the intended screen layout and information architecture described in Section 5; minor branding and wording differences from the deployed application may exist.

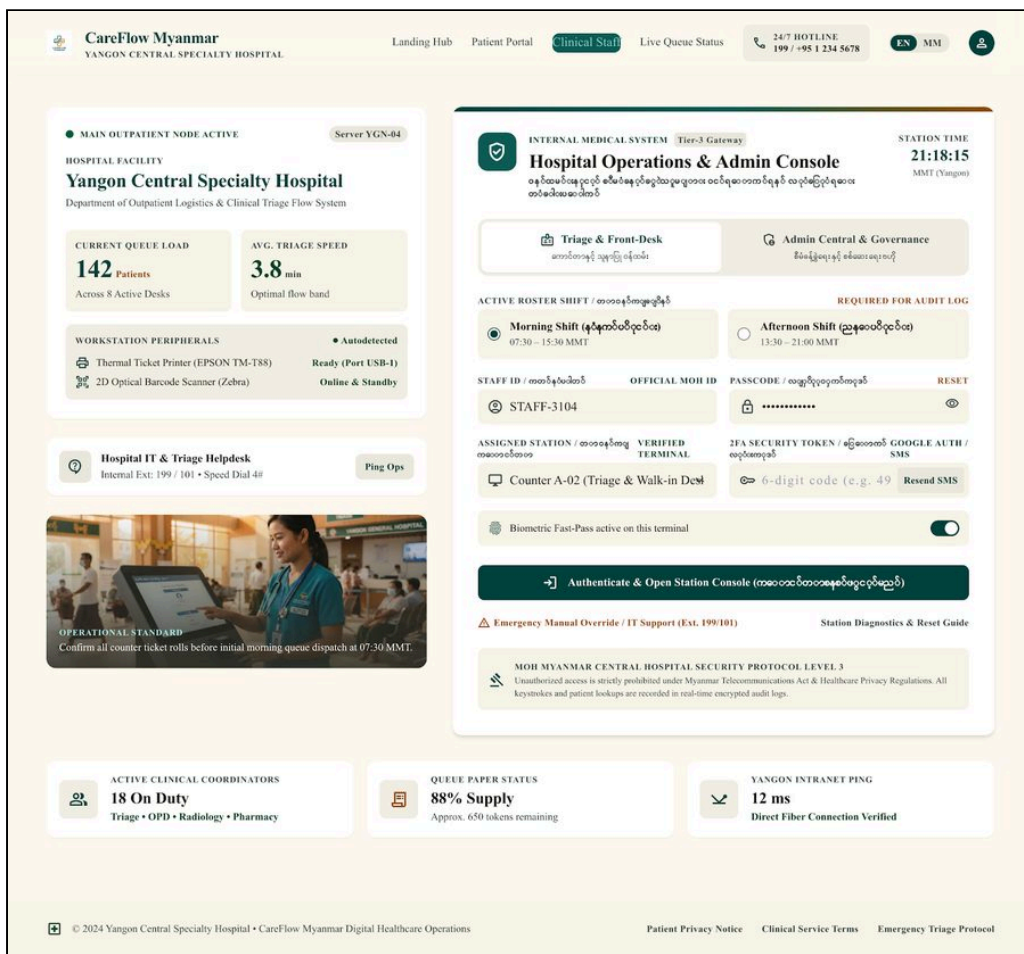
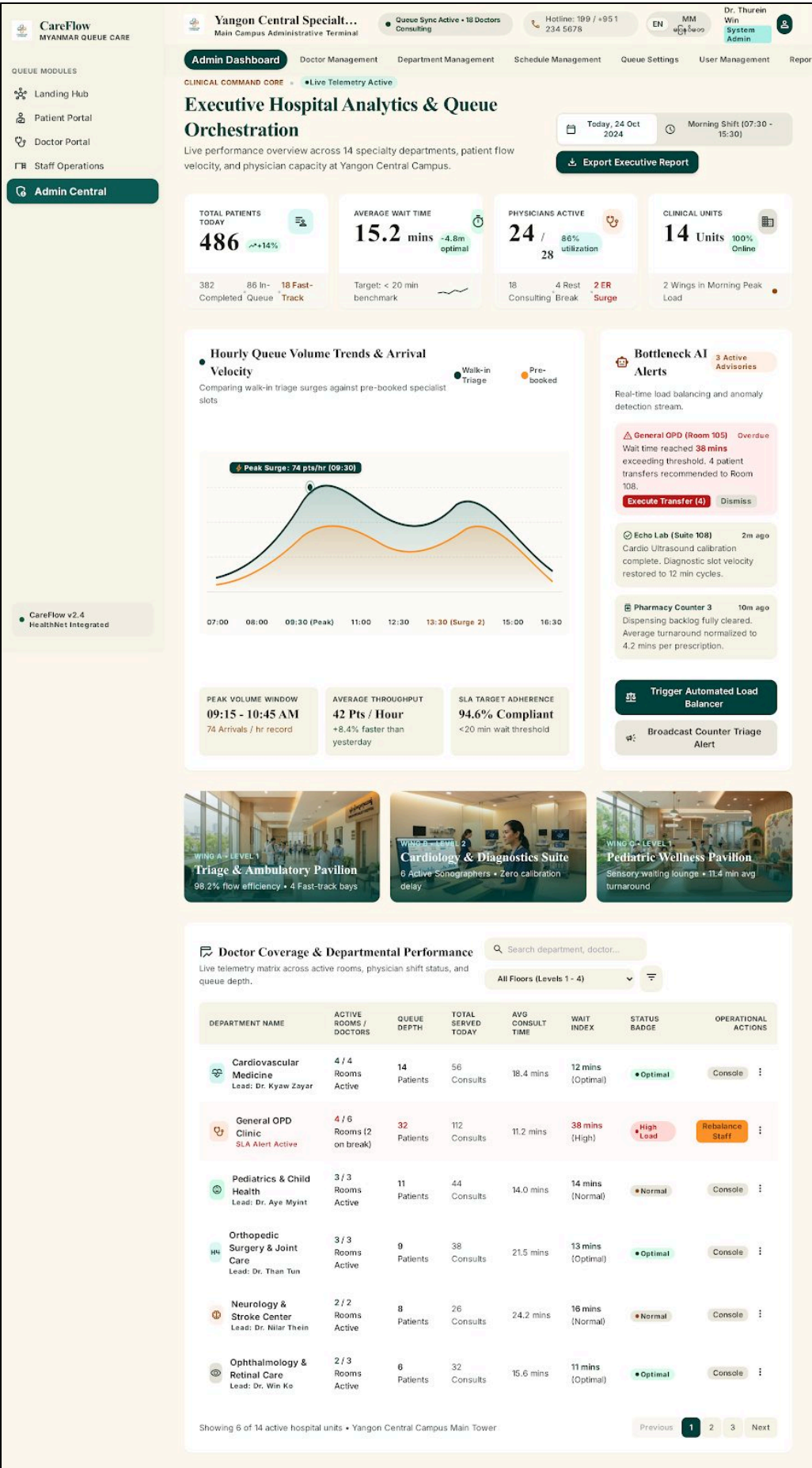


Figure 1: Hospital Smart Queue System — Login Page



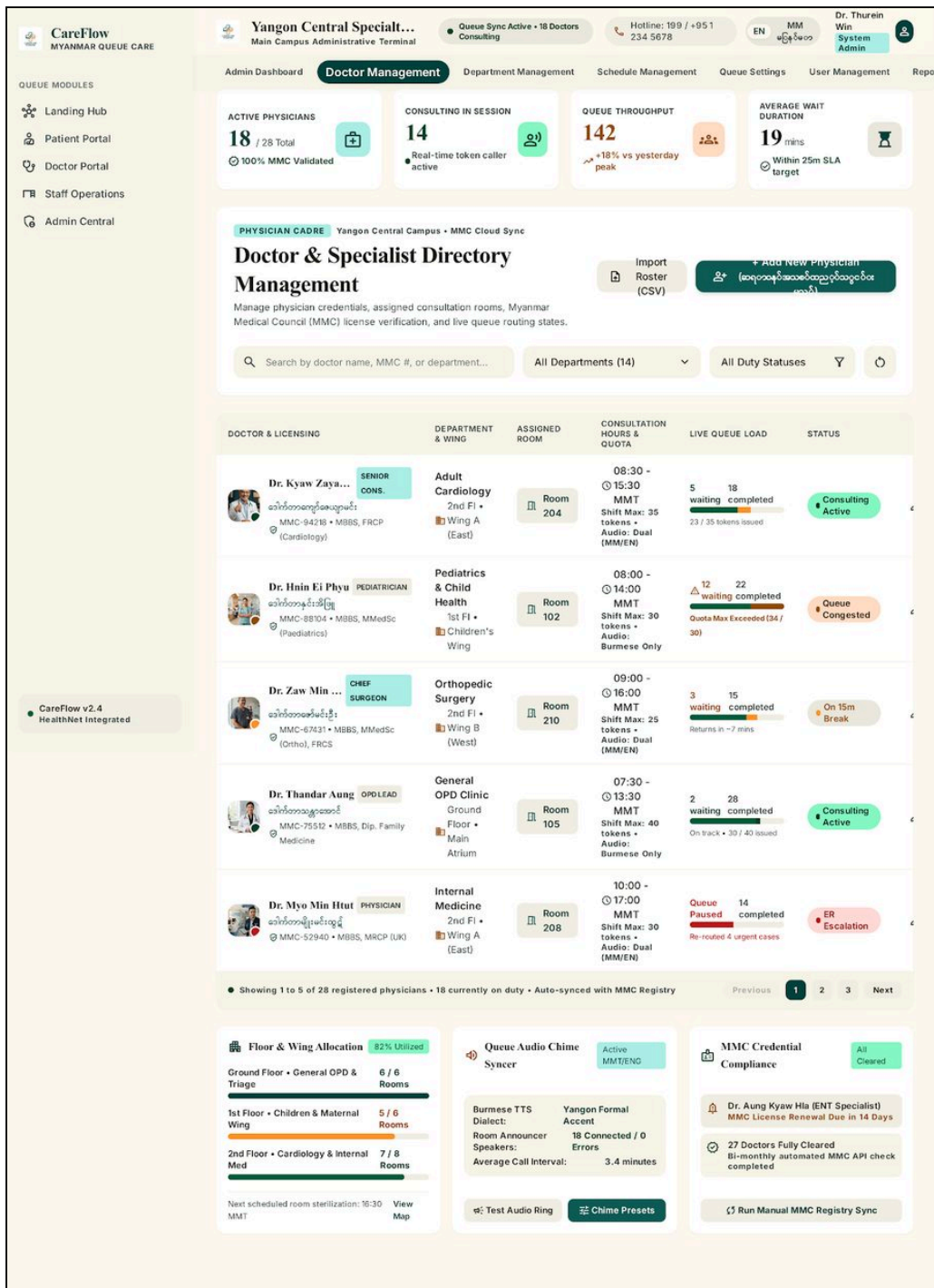


Figure 4: Doctor Management Page

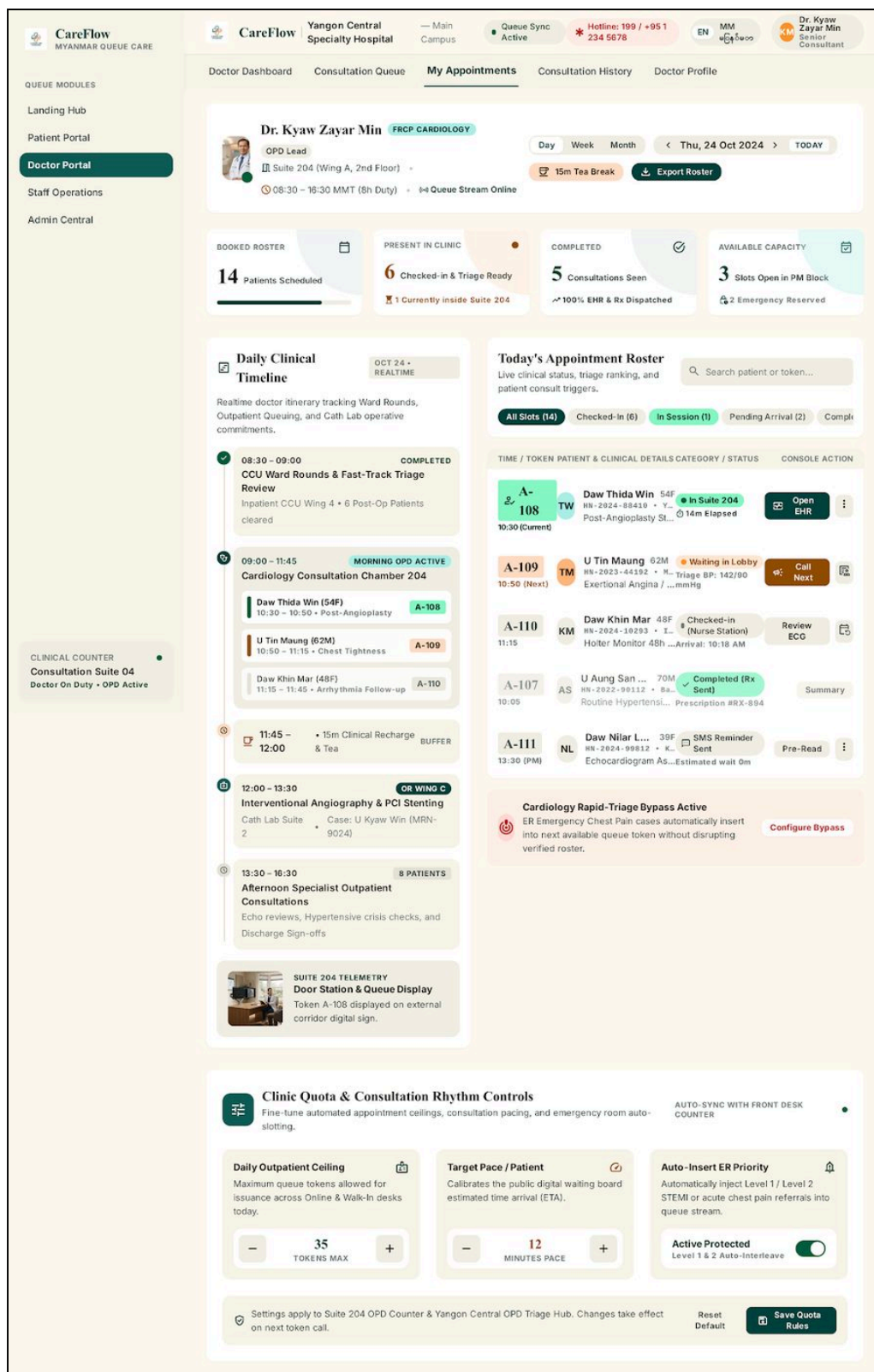


Figure 5: Appointment Management Page

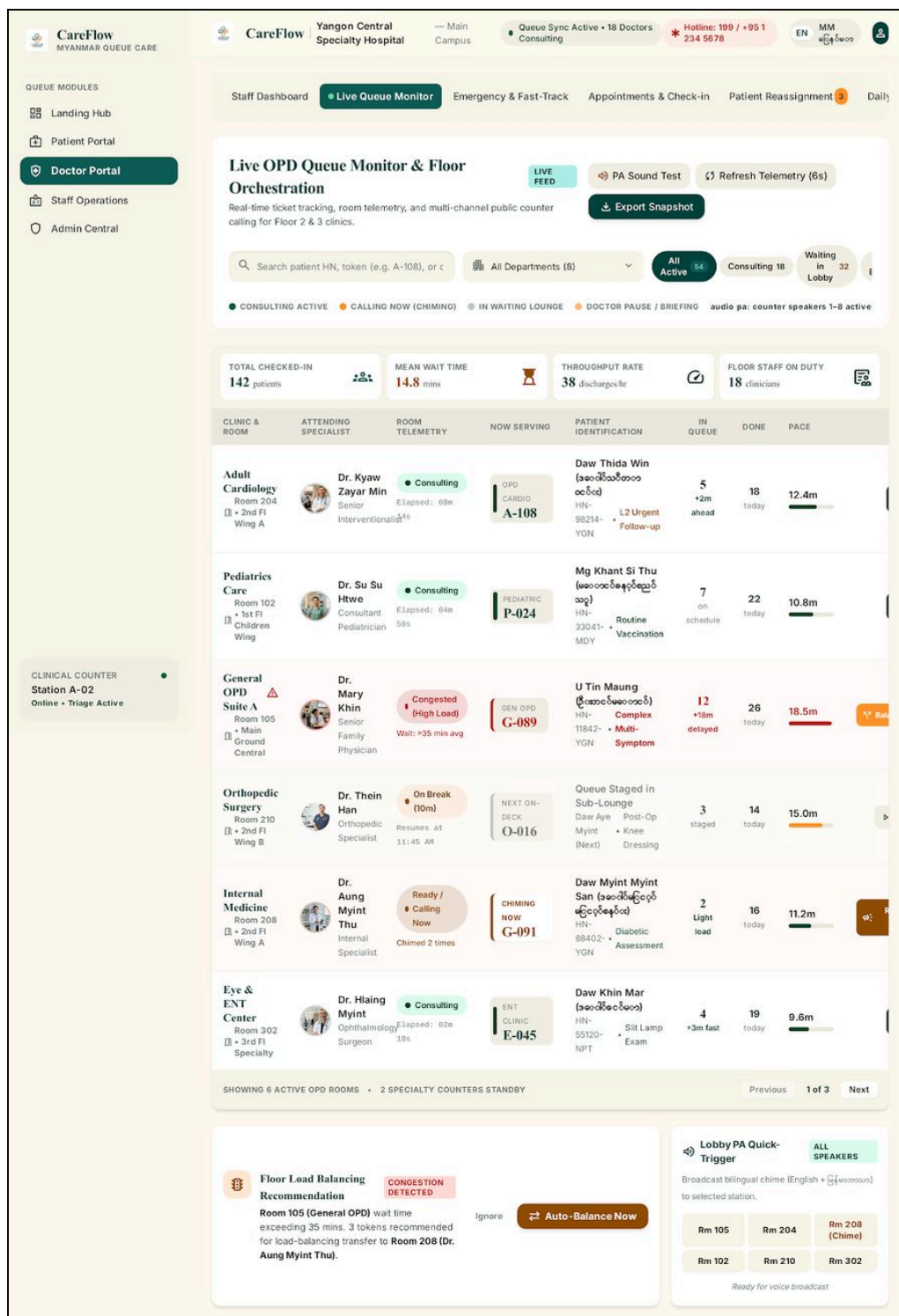


Figure 6: Staff Queue Management Page

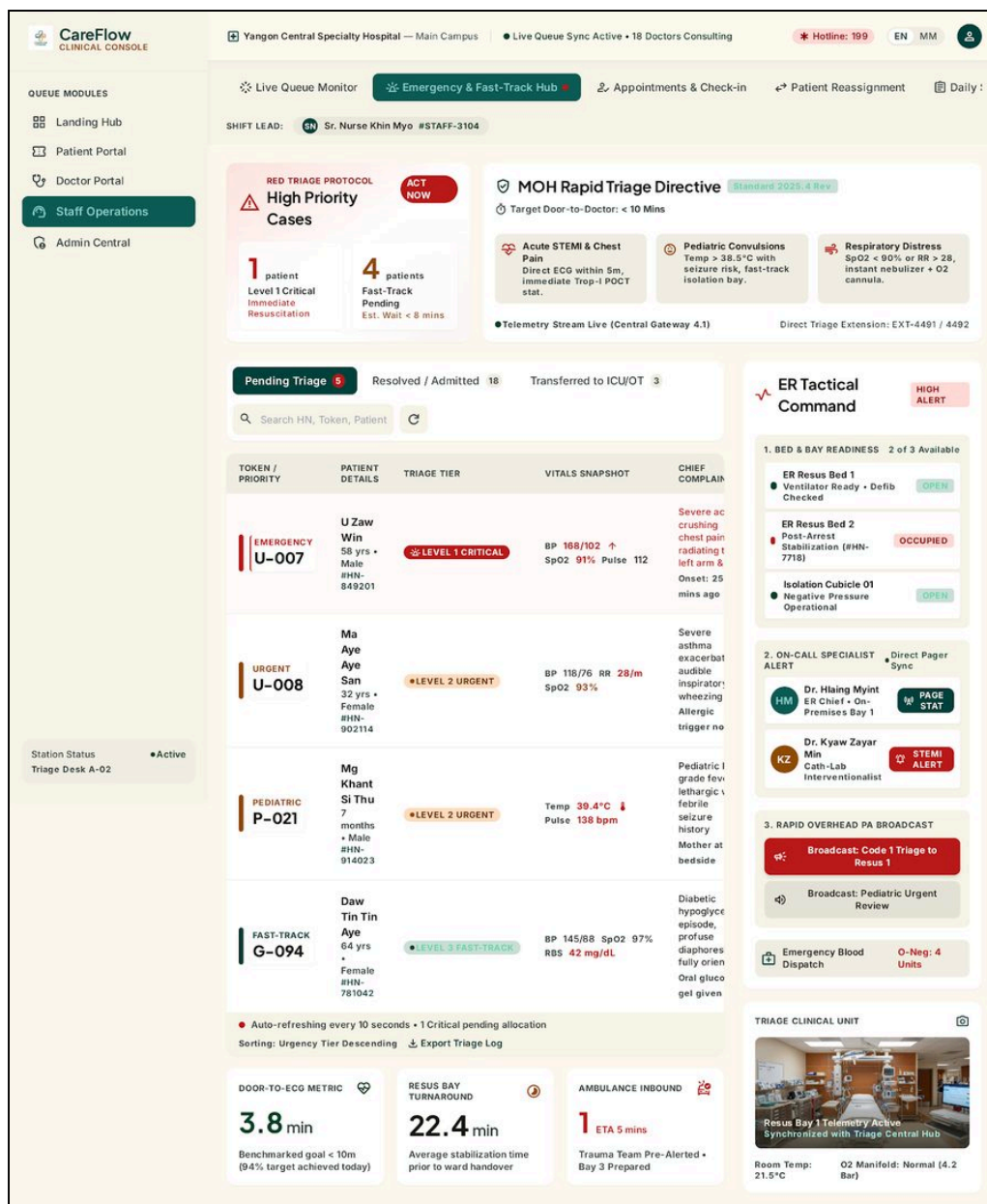


Figure 7: Emergency Case Management Page

CareFlow

CLINICAL CONSOLE

Yangon Central Specialty Hospital — Main Campus

Live Queue Sync Active • 18 Doctors Consulting

Hotline: 199

EN

MM

QUEUE MODULES

Landing Hub

Patient Portal

Doctor Portal

Staff Operations

Admin Central

Dr. Kyaw Zayar Min

Senior Consultant Cardiologist

Cardiology Dept

OPD ROOM 204

License: MMC-49102

Digital Sign Active

TODAY'S LOAD

15 / 22 Seen

On Duty

Accepting Patients

DESK AUTOMATION QUEUE SWITCH

Call Next Patient (A-108)

Start Consultation

Room 204

Intercom Feed

AUTO-SYNC ON

Pause / Break (15m)

Escalate / ER

TOKEN

A-108

TRIAGE SLOT

Cardio

Reg 02

Normal OPD

54 Yrs • Female • Blood Group: O Rh+

HN: #YGN-849201

ARRIVAL

10:15 AM

Waited 22 mins

CHIEF COMPLAINT & TRIAGE NOTES

"Recurrent retrosternal chest discomfort, mild tightness, and unusual fatigue after climbing 1 flight of stairs, ongoing for 4 days. No syncope."

Reported by: Staff Nurse Hnin

Previous Cardiac History: Mild Hypertension (2 yrs)

RECORDED VITALS (10:18 AM)

BP (MMHG)

135/88

Stage 1 Elevated

PULSE (BPM)

78

Normal Sinus

SPO2 (%)

98%

Room Air

TEMP

36.8°C

Afebrile

WEIGHT

62.0 kg

BMI: 24.2 (Normal)

Verified by Omron Triage Station

View Past EMR (3 Visits)

Past Labs & Echo

Attending Nurse: Sister May Thu

Room 204 Queue

Waiting (7) Called (1) Done (14)

A-109

U Aung Myo Zaw

Priority Cardiac

Waited 38m

Call Now

A-109

PRIORITY

38m in lobby

U Aung Myo Zaw

Post-PCI Follow-up (67M)

BP: 142/90 • HR: 64

Select >

A-110

OPD REGULAR

19m in lobby

Daw Khin San Tint

Dyslipidemia review (48F)

BP: 122/80 • HR: 72

Select >

A-111

REFILL ONLY

14m in lobby

U Tin Htut Oo (61M)

Warfarin INR 2.4

BP: 128/82 • INR Stable

Select >

A-112

NEW PATIENT

9m in lobby

Ma Zin Mar Lwin (32F)

Palpitation workup

BP: 118/76 • HR: 88

Select >

Audio Paging

Chime

Bell 1

Burmese Voice

AVERAGE CONSULTATION SPEED

11.4 min / patient

Well within target threshold (< 15 mins)

CareFlow Clinical Copilot

LLM V4.2 MEDICAL

Synthesized from Triage Vitals, OPD Notes & Regional Guideline (AHA/ACC 2024)

CLINICAL REASONING PROMPT

Patient presents with exertion-linked retrosternal discomfort and borderline systolic blood pressure (135/88 mmHg). High suspicion index for **Stable Angina** vs early atypical ischemic symptoms. Musculoskeletal chest wall pain remains secondary differential due to absence of chest tenderness in triage nurse notes.

Automated SOAP Note Draft

Ready for Physician Verification

Subjective

Daw Thida Win, 54 yo female, complains of recurrent retrosternal heaviness radiating mildly to left shoulder upon climbing stairs or carrying groceries for past 4 days. Episodes last 5-10 mins, relieved by sitting/ resting. Denies diaphoresis, palpitation, or orthopnea. Known history of hypertension treated inconsistently with Amlodipine 5mg.

Objective

BP: 135/88 mmHg, HR: 78 bpm regular, SpO2: 98% room air, Afebrile (36.8°C). Alert, oriented x3, no respiratory distress sitting. Heart: S1/S2 present, no murmurs/gallops heard. Lungs: Clear to auscultation bilaterally. No peripheral pedal edema. Chest wall palpation non-tender.

Assessment & ICD-10 Coding

Probable Stable Angina Pectoris (CCS Class II) secondary to coronary artery insufficiency; essential hypertension.

I20.9 • Angina pectoris, unspecified

88% Match

R07.9 • Chest pain, unspecified

54% Match

I10 • Essential (primary) hypertension

Treatment & Diagnostic Plan

1. Immediate Diagnostics:

12-Lead Resting ECG in OPD Room 204; Stat Troponin-I POCT; Fasting Lipid Panel & HbA1c requisition.

2. Therapeutic Prescriptions:

Aspirin 81mg daily; Sublingual Nitroglycerin 0.5mg PRN for acute chest pressure; Adjust Amlodipine 5mg to 10mg daily.

3. Next Step:

Schedule outpatient Treadmill Exercise Stress Test (TMT) within 72 hours; follow up in clinic 14 days.

Approve & Insert into EHR

Copy SOAP Note

Regenerate Draft

Figure 8: Doctor Consultation Page

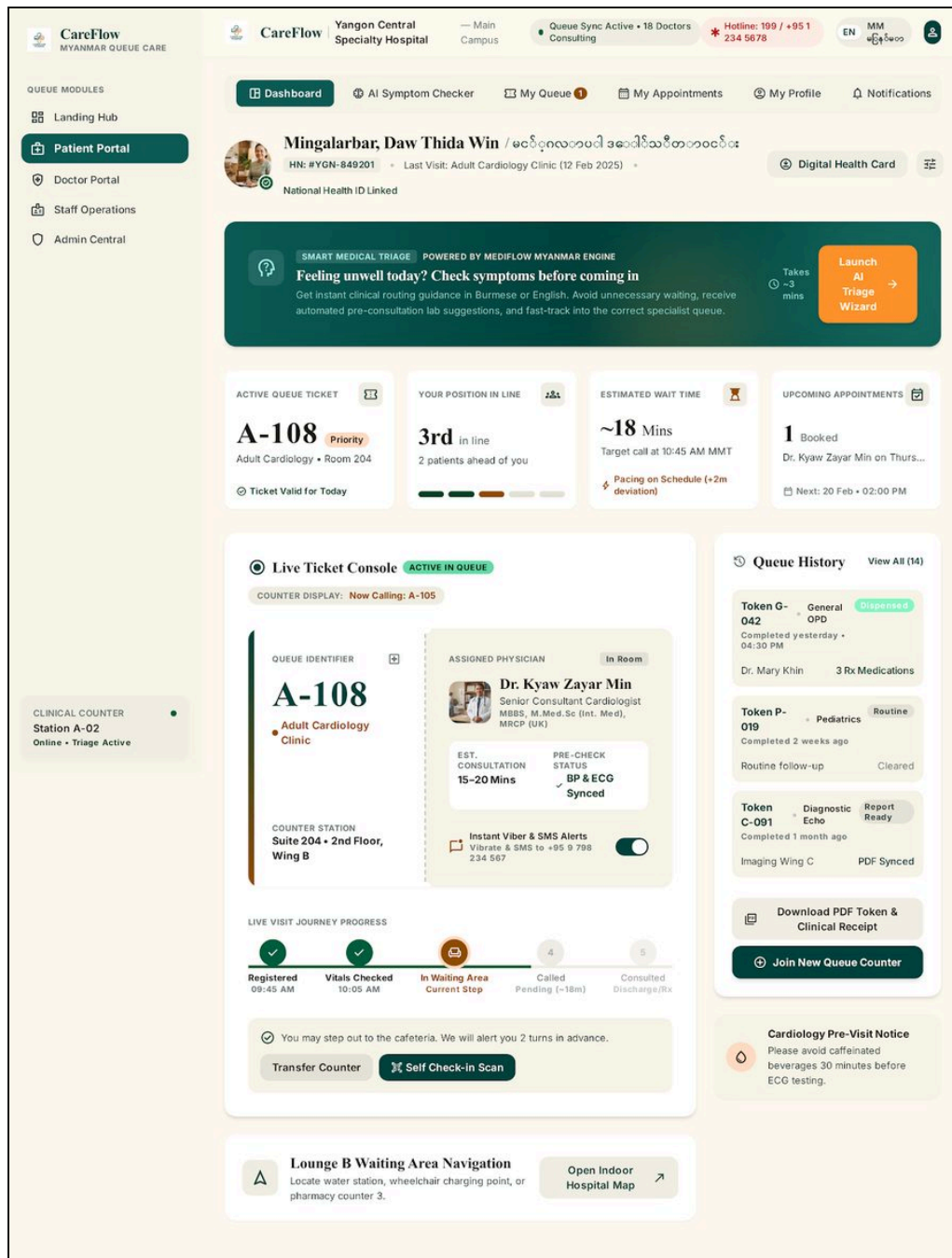


Figure 9: Patient Dashboard

7.2 References

- [1] L. V. Green, "Queueing analysis in healthcare," in Patient Flow: Reducing Delay in Healthcare Delivery, R. W. Hall, Ed. Boston, MA: Springer, 2006, pp. 281–307.
- [2] N. Gilboy, P. Tanabe, D. Travers, and A. M. Rosenau, Emergency Severity Index (ESI): A Triage Tool for Emergency Department Care, Version 4. Implementation Handbook 2012 Edition, AHRQ Pub. No. 12-0014, Rockville, MD: Agency for Healthcare Research and Quality, 2012.

- [3] F. Y. Lin and H. C. Chen, "Machine learning algorithms for predicting outpatient waiting time," *International Journal of Environmental Research and Public Health*, vol. 17, no. 18, p. 6542, 2020.
- [4] I. Mohammadi, H. Wu, A. Turkcan, T. Toscos, and B. N. Doebbeling, "Data analytics and modeling for appointment no-show in outpatient clinics," *Journal of Healthcare Engineering*, vol. 2018, Article ID 4187050, 2018.
- [5] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS 2020)*, vol. 33, pp. 9459–9474, 2020.
- [6] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA: Prentice Hall, 2017.
- [7] J. Bloch, *Effective Java*, 3rd ed. Boston, MA: Addison-Wesley Professional, 2018.
- [8] C. Walls, *Spring in Action*, 6th ed. Shelter Island, NY: Manning Publications, 2022.
- [9] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York: McGraw-Hill Education, 2020.
- [10] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Department of Information and Computer Science, University of California, Irvine, CA, 2000.